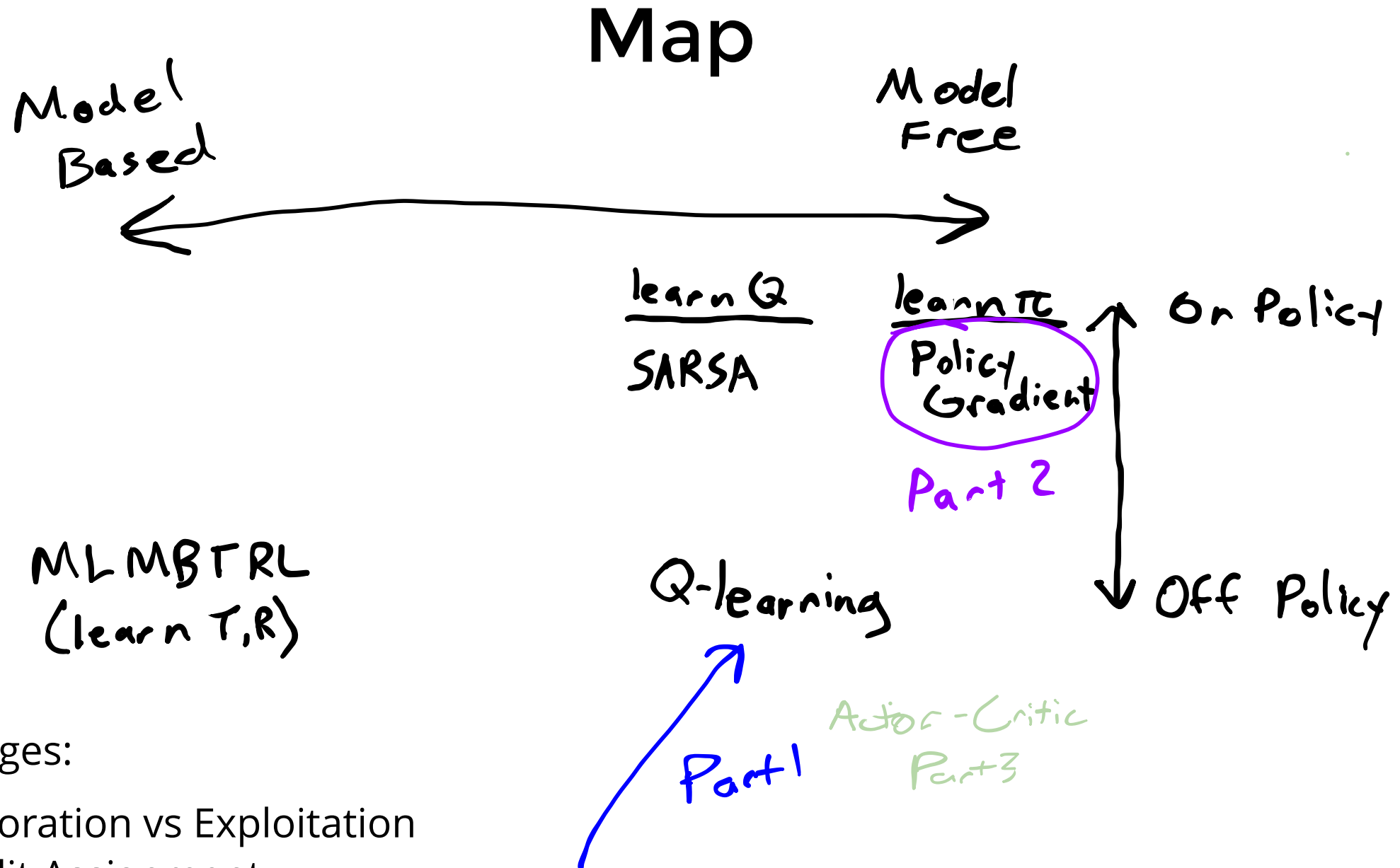


DQN and Advanced Policy Gradient

Map



Challenges:

1. Exploration vs Exploitation
2. Credit Assignment
3. Generalization

Last Time: Neural Networks

Part I

DQN

Q-Learning with Neural Networks

Q-Learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

Neural Networks

$$\theta^* = \arg \min_{\theta} \sum_{(x,y) \in \mathcal{D}} l(f_{\theta}(x), y)$$

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} l(f_{\theta}(x), y)$$

Deep Q learning:

- Approximate Q with Q_{θ}
- What should (x, y) be?
- What should l be?

Candidate Algorithm:

loop

$$a \leftarrow \operatorname{argmax} Q(s, a) \text{ w.p. } 1 - \epsilon, \quad \text{rand}(A) \text{ o.w.}$$

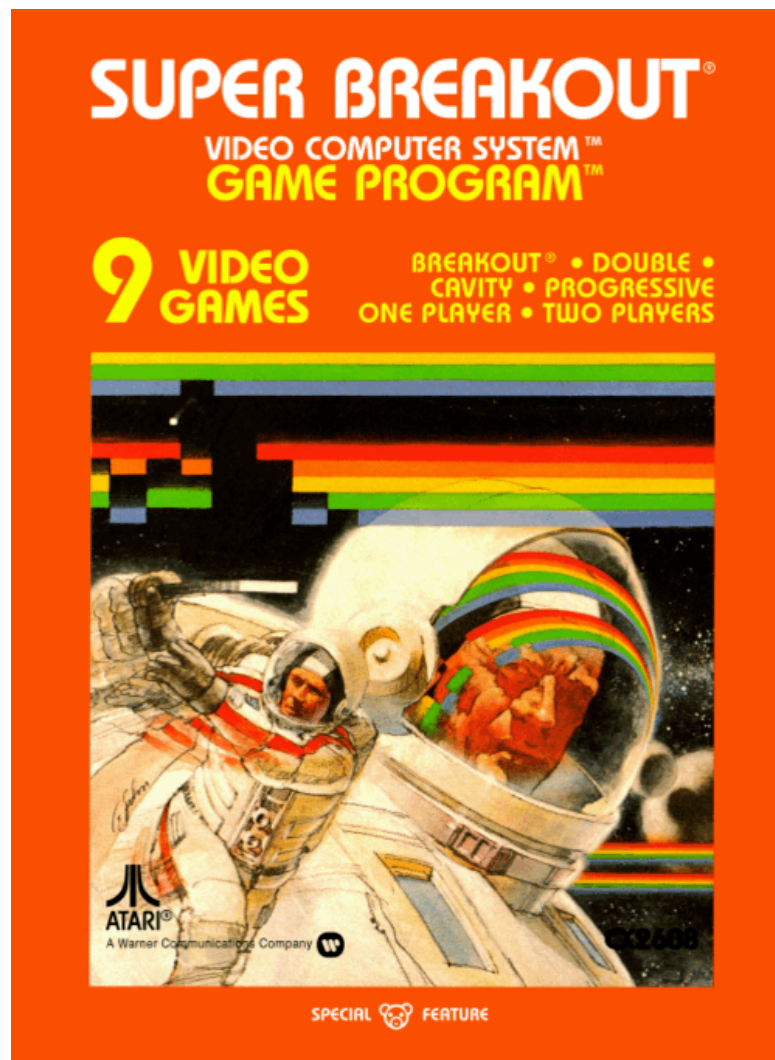
$$r \leftarrow \text{act!}(\text{env}, a)$$

$$s' \leftarrow \text{observe}(\text{env})$$

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} (r + \gamma \max_{a'} Q_{\theta}(s', a') - Q_{\theta}(s, a))^2$$

$$s \leftarrow s'$$

DQN: The Atari Benchmark



DQN: Problems with Naive Approach

Candidate Algorithm:

loop

$a \leftarrow \operatorname{argmax} Q(s, a) \text{ w.p. } 1 - \epsilon, \quad \operatorname{rand}(A) \text{ o.w.}$

$r \leftarrow \operatorname{act}!(\operatorname{env}, a)$

$s' \leftarrow \operatorname{observe}(\operatorname{env})$

$\theta \leftarrow \theta - \alpha \nabla_{\theta} (r + \gamma \max_{a'} Q_{\theta}(s', a') - Q_{\theta}(s, a))^2$

$s \leftarrow s'$

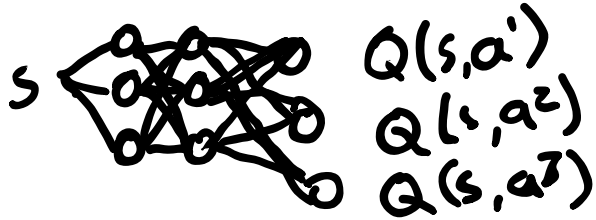
Problems:

1. Samples Highly Correlated
2. Size-1 batches
3. Moving target

} data buffer / experience replay
} periodically freeze target

DQN

Q Network Structure:



Experience Tuple: (s, a, r, s')

Loss:

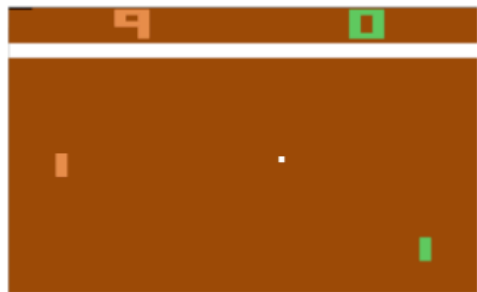
$$l(s, a, r, s') = \left(r + \gamma \max_{a'} Q_{\theta'}(s', a') - Q_{\theta}(s, a) \right)^2$$

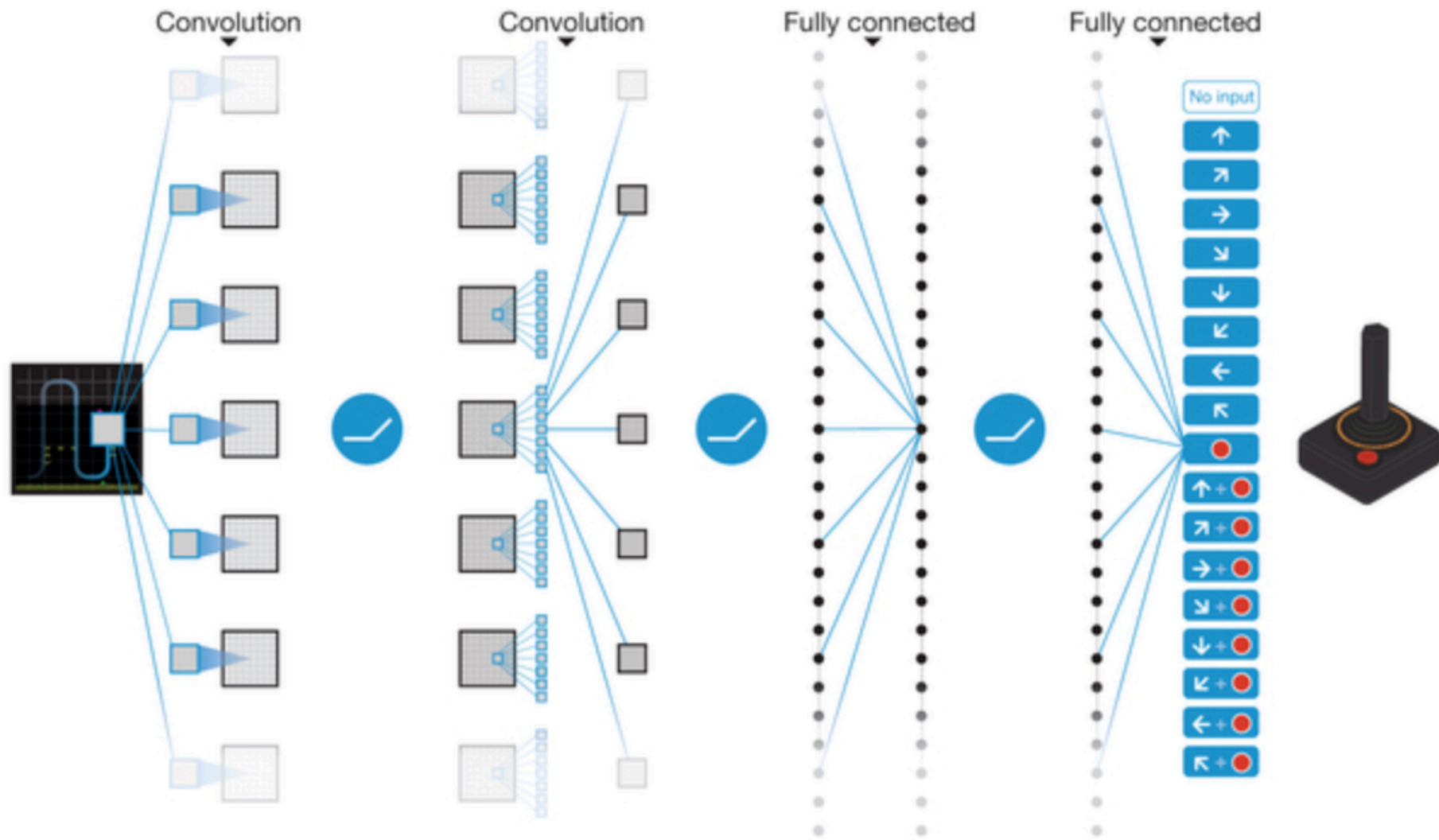
Playing Atari with Deep Reinforcement Learning

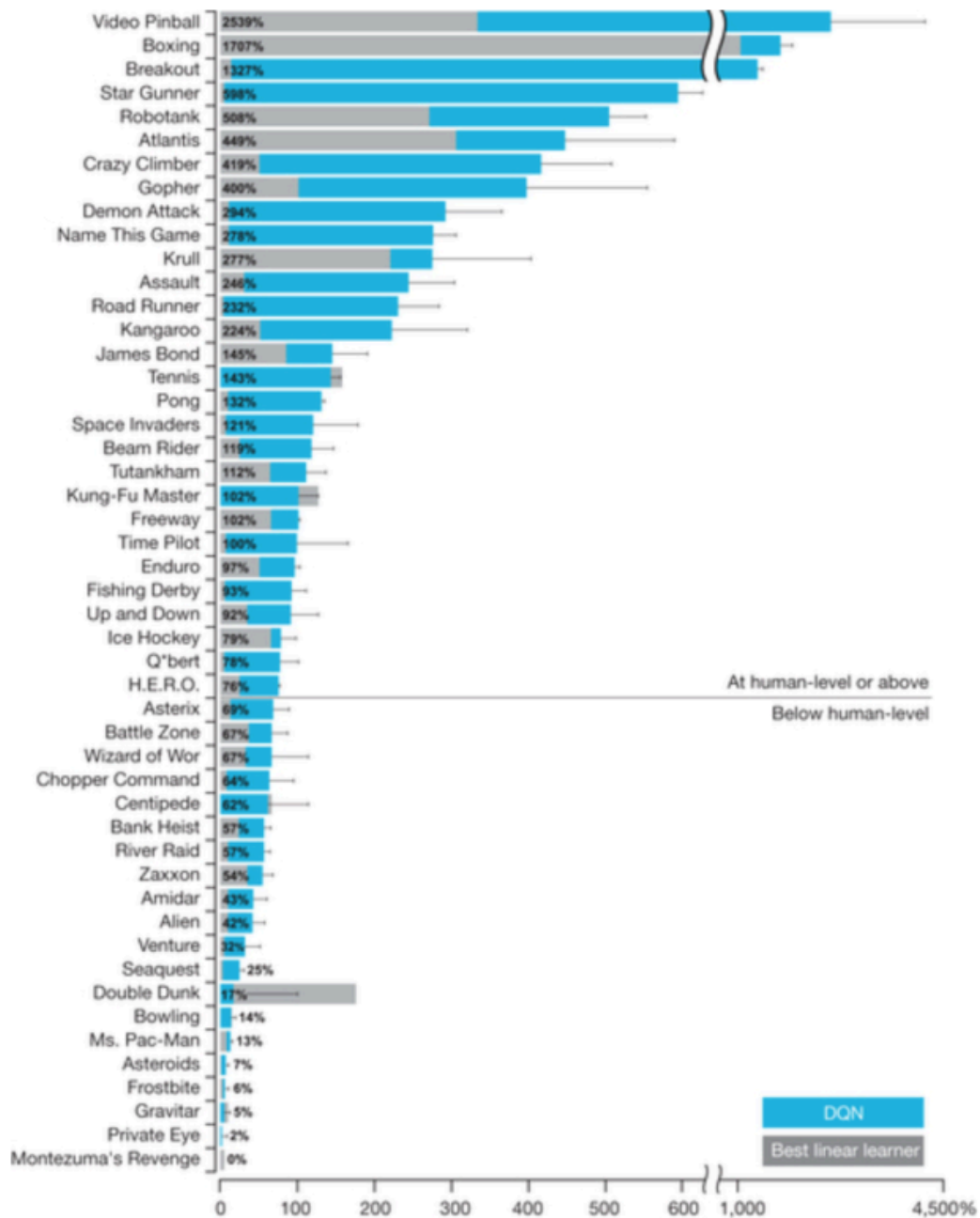
Volodymyr Mnih Koray Kavukcuoglu David Silver Alex Graves Ioannis Antonoglou

Daan Wierstra Martin Riedmiller

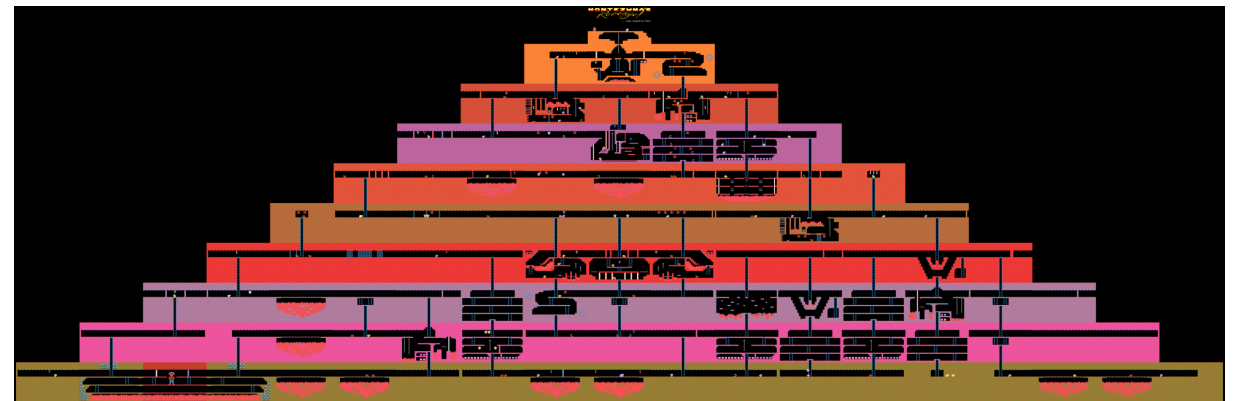
DeepMind Technologies







<https://www.youtube.com/watch?v=SuZVyOlgVek>



Rainbow

- Double Q Learning
- Prioritized Replay
(priority proportional to last TD error)
- Dueling networks

Value network + advantage network

$$Q(s, a) = V(s) + A(s, a)$$

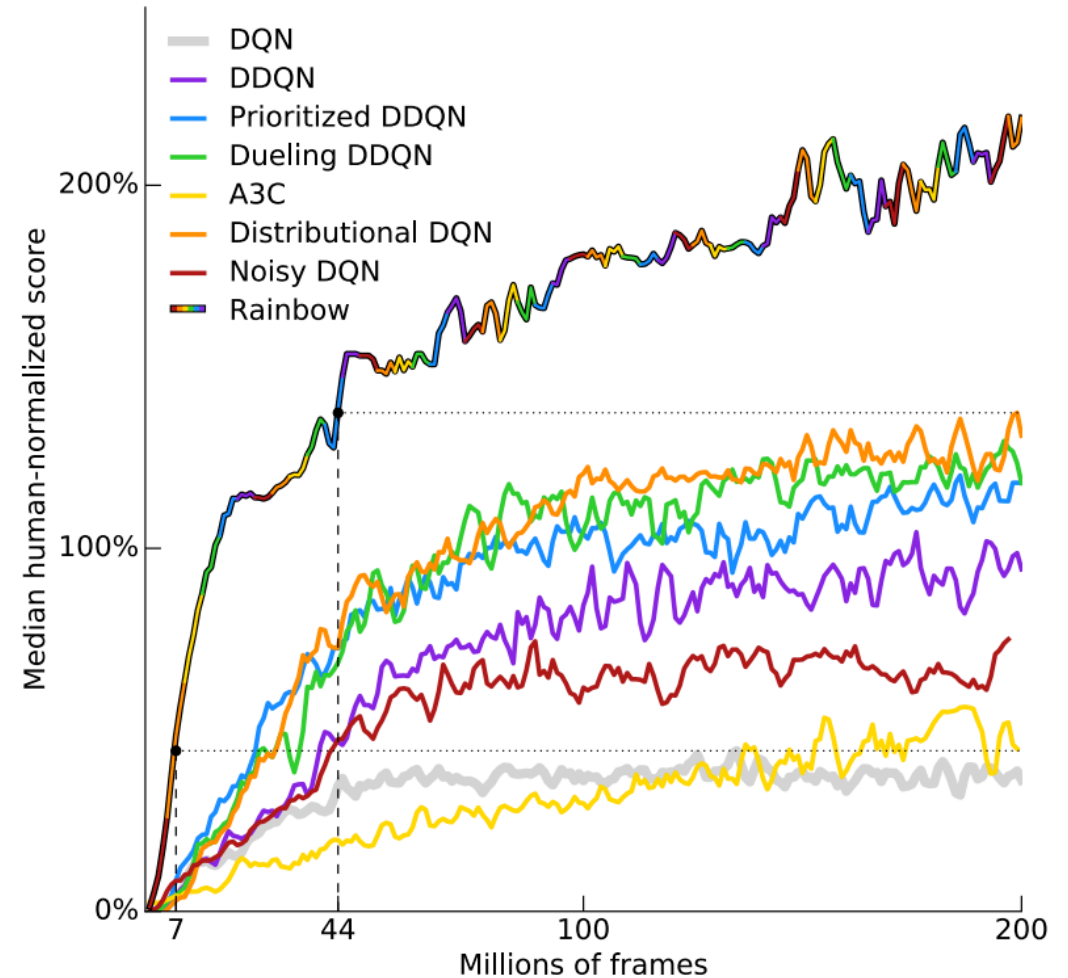
- Multi-step learning

$$(r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \gamma \max Q_{\theta}(s_{t+n}, a') - Q_{\theta}(s_t, a_t))^2$$

- Distributional RL

predict an entire distribution of values instead of just Q

- Noisy Nets



Actual Learning Curves

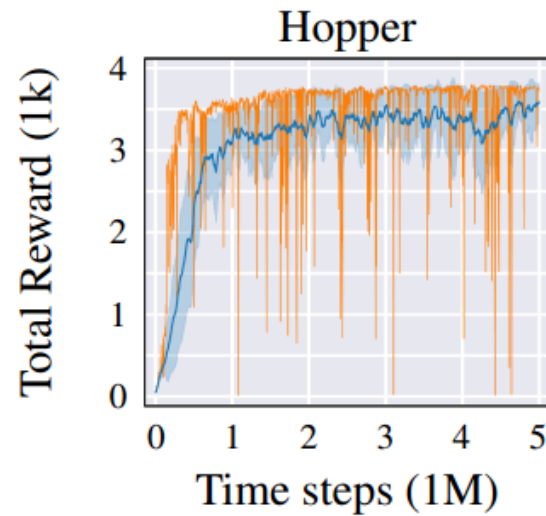


Figure 24: ■ Standard presentation ■ Single seed

The training curve as commonly presented in RL papers ■ and the training curve of a single seed ■. Both curves are from the TD3 algorithm, trained for 5M time steps. The standard presentation is to evaluate every N_{freq} steps, average scores over N_{episodes} evaluation episodes and N_{seeds} seeds, then to smooth the curve by averaging over a window of N_{window} evaluations. (In our case this corresponds to $N_{\text{freq}} = 5000$, $N_{\text{episodes}} = 10$, $N_{\text{seeds}} = 10$, $N_{\text{window}} = 10$). The learning curve of a single seed has no smoothing over seeds or evaluations ($N_{\text{seeds}} = 1$, $N_{\text{window}} = 1$). By averaging over many seeds and evaluations, the training curves in RL can appear deceptively smooth and stable.

Paper: [For SALE: State-Action Representation Learning for Deep Reinforcement Learning](#)

Part II

Improved Policy Gradients

Policy Gradient

$$\widehat{\nabla_{\theta} U(\theta)} = \sum_{k=0}^d \nabla_{\theta} \log \pi_{\theta}(a_k \mid s_k) \gamma^k (r_{k,\text{to-go}} - r_{\text{base}}(s_k))$$

$\uparrow$ $\uparrow$
 $\hat{Q}^{\pi}(s_k, a_k)$ $\hat{U}^{\pi}(s_k)$

Advantage Function:
 $A(s, a) \equiv Q(s, a) - U(s)$

loop:

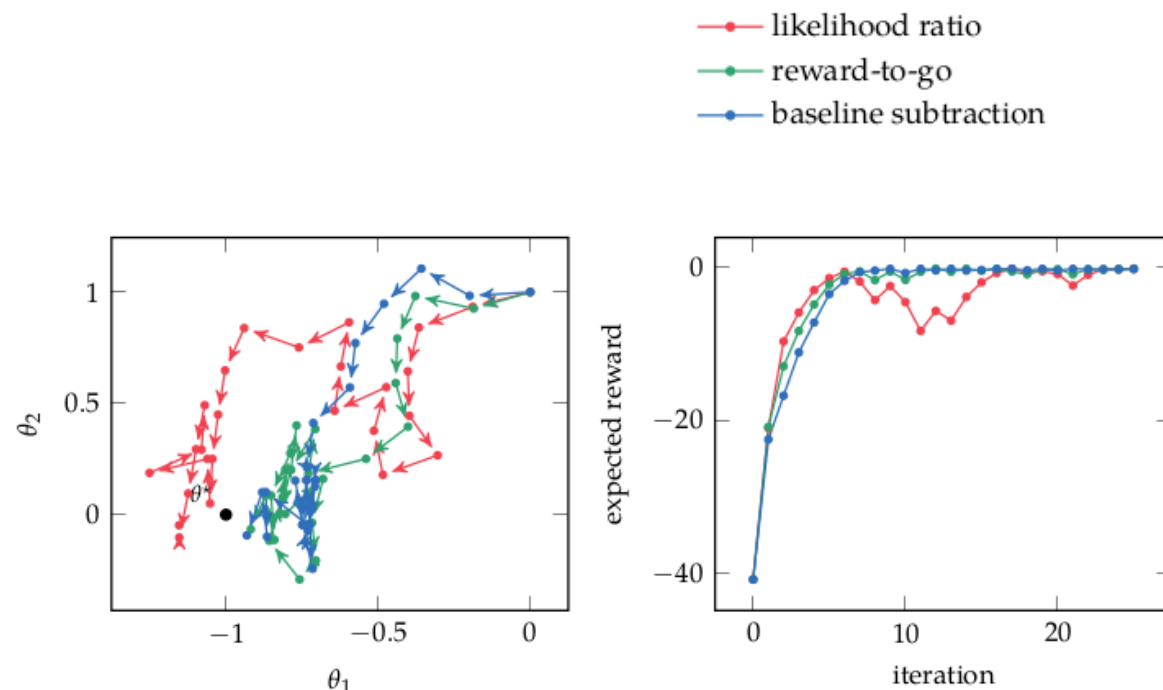
$\tau \leftarrow \text{simulate_episode}(\pi_{\theta})$

$\widehat{\nabla_{\theta} U(\theta)} \leftarrow \sum_{k=0}^d \nabla_{\theta} \log \pi_{\theta}(a_k \mid s_k) \gamma^k \hat{A}_k$

$\theta \leftarrow \theta + \alpha \widehat{\nabla_{\theta} U(\theta)}$

Some problems:

- $\widehat{\nabla_{\theta} U(\theta)}$ has high variance
- The gradient is not a good step direction
- Fixed α steps are too big or small



Trust Region Optimization

$$\widehat{\nabla U}(\theta) = \sum_{k=0}^d \nabla_{\theta} \log \pi_{\theta}(a_k \mid s_k) \gamma^k \hat{A}_k$$

$$\theta' = \theta + \alpha \widehat{\nabla U}(\theta)$$

Example:

$$g(\theta, \theta') = \|\theta - \theta'\|_2^2 = \frac{1}{2}(\theta' - \theta)^{\top}(\theta' - \theta)$$

$$\underset{\theta'}{\text{maximize}} \quad U(\theta') \approx U(\theta) + \nabla U(\theta)^{\top}(\theta' - \theta)$$

$$\text{subject to} \quad g(\theta, \theta') \leq \epsilon$$

$$\theta' = \theta + \mathbf{u} \sqrt{\frac{2\epsilon}{\mathbf{u}^{\top} \mathbf{u}}} = \theta + \sqrt{2\epsilon} \frac{\mathbf{u}}{\|\mathbf{u}\|}$$

$$\mathbf{u} = \nabla U(\theta)$$

Natural Gradient

$$g(\theta, \theta') = D_{\text{KL}}(p(\cdot | \theta) || p(\cdot | \theta')) \leq \epsilon$$

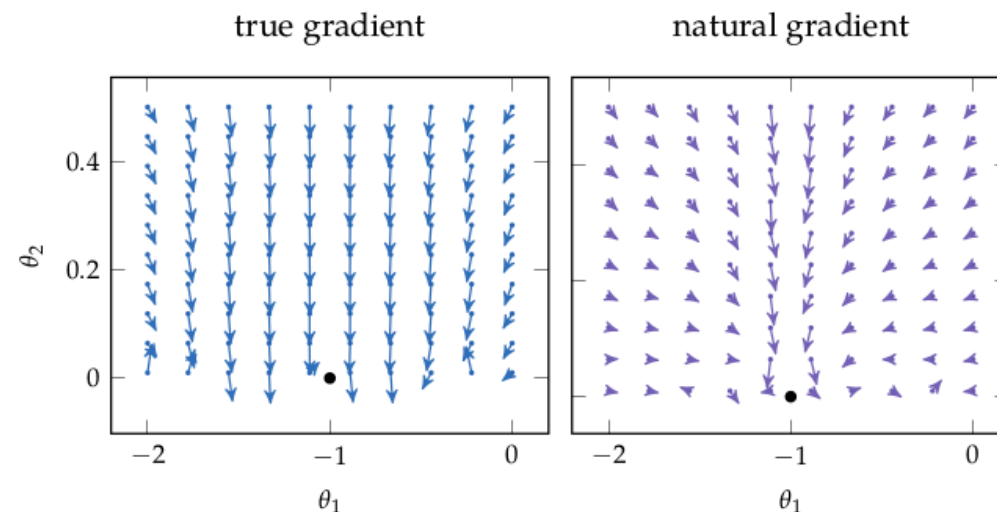
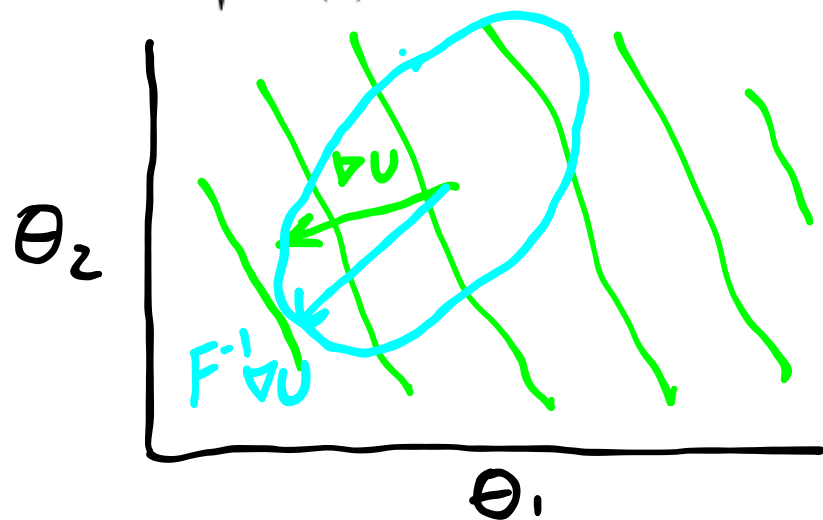
$$D_{\text{KL}}(p || q) = \int p(x) \log \frac{p(x)}{q(x)} dx = - \int p(x) \log \frac{q(x)}{p(x)} dx$$

$$g(\theta, \theta') = \frac{1}{2}(\theta' - \theta)^\top \mathbf{F}_\theta (\theta' - \theta) \leq \epsilon$$

$\mathcal{L}$ Fischer Information Matrix

$$\begin{aligned} \mathbf{F}_\theta &= \int p(\tau | \theta) \nabla \log p(\tau | \theta) \nabla \log p(\tau | \theta)^\top d\tau \\ &= \mathbb{E}_\tau \left[\nabla \log p(\tau | \theta) \nabla \log p(\tau | \theta)^\top \right] \end{aligned}$$

$$\theta' = \theta + \mathbf{u} \sqrt{\frac{2\epsilon}{\nabla U(\theta)^\top \mathbf{u}}} \quad \mathbf{u} = \mathbf{F}_\theta^{-1} \nabla U(\theta)$$



Conservative Policy Iteration Objective

Discounted Visitation Distribution

$$E_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] = E_{\substack{s \sim b_{\gamma, \theta} \\ a \sim \pi_\theta(\cdot | s)}} [R(s, a)]$$

$$b_{\gamma, \theta}(s) \propto P(s_1 = s) + \gamma P(s_2 = s) + \gamma^2 P(s_3 = s) + \dots$$

Policy Advantage

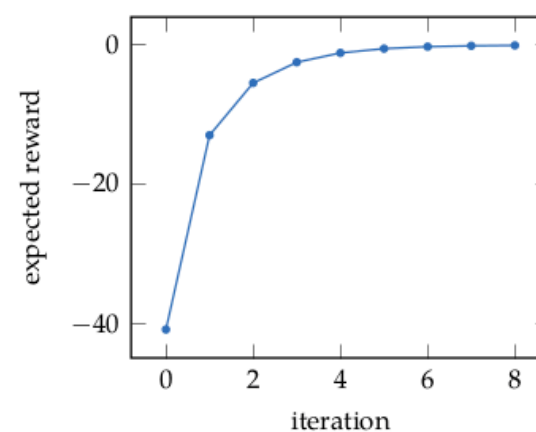
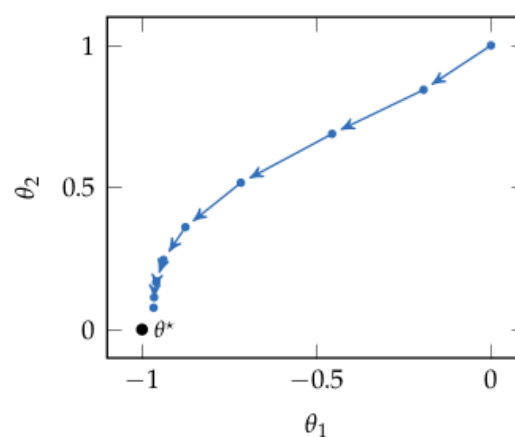
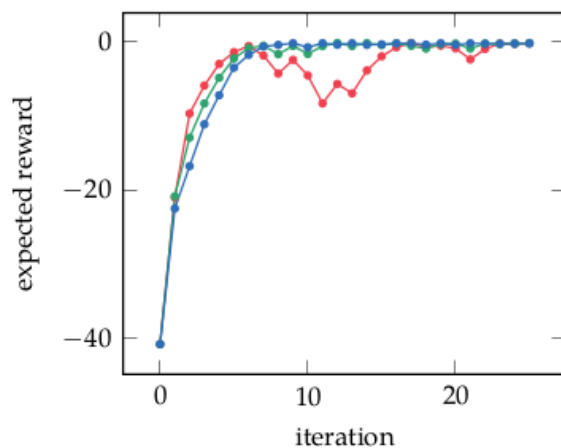
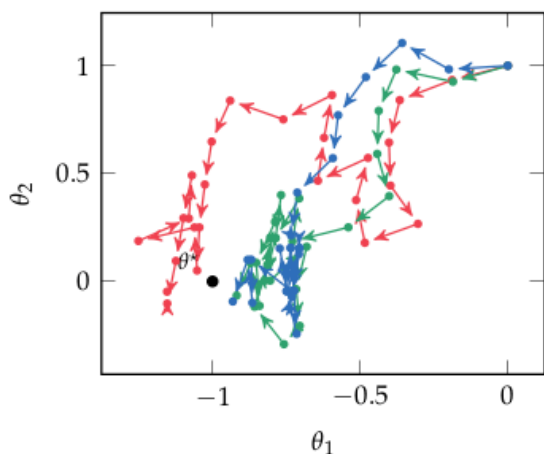
"Approximately Optimal Approximate RL"
Kakade & Langford

Trust Region Policy Optimization (TRPO)

Natural Gradient Approximation + Conservative Policy Iteration Objective

Update Step:

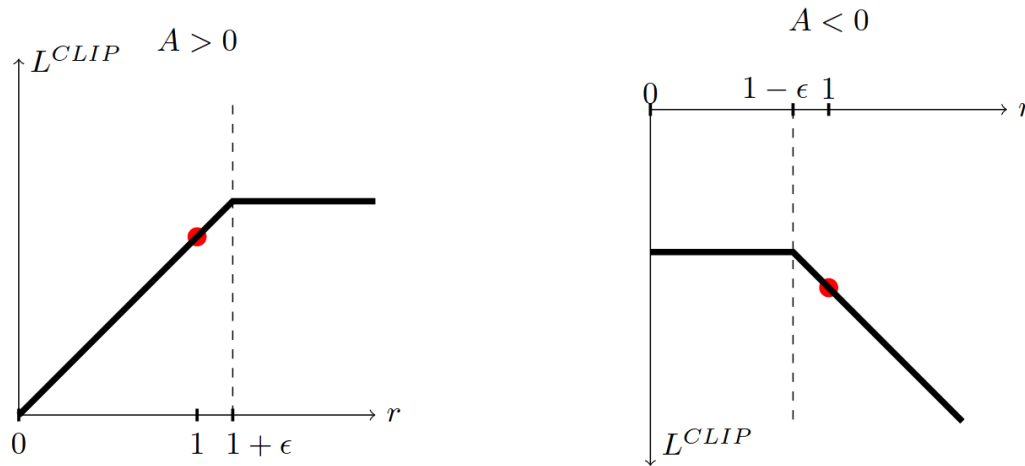
$$\begin{aligned} \underset{\theta'}{\text{maximize}} \quad & f(\theta, \theta') = E_{\substack{s \sim b_{\gamma, \theta} \\ a \sim \pi_{\theta}(\cdot | s)}} \left[\frac{\pi_{\theta'}(a | s)}{\pi_{\theta}(a | s)} A^{\pi_{\theta}}(s, a) \right] \\ \text{subject to} \quad & g(\theta, \theta') = \mathbb{E}_{s \sim b_{\gamma, \theta}} [D_{\text{KL}}(\pi_{\theta}(\cdot | s) \parallel \pi_{\theta'}(\cdot | s))] \leq \epsilon \end{aligned}$$



Proximal Policy Optimization (PPO)

Clipped Surrogate Objective:

$$E_{\substack{s \sim b_{\gamma, \theta} \\ a \sim \pi_{\theta}(\cdot | s)}} \left[\min \left(\frac{\pi_{\theta'}(a | s)}{\pi_{\theta}(a | s)} A_{\theta}(s, a), \text{clamp} \left(\frac{\pi_{\theta'}(a | s)}{\pi_{\theta}(a | s)}, 1 - \epsilon, 1 + \epsilon \right) A_{\theta}(s, a) \right) \right]$$



$$r = \frac{\pi_{\theta'}(a | s)}{\pi_{\theta}(a | s)}$$

Each step: take several gradient steps with trajectories from π_{θ}
(No need for constraint because it will stop when it hits the clip)

PPO Impact

Cited 29262 times (as Gall 2025)

(avg ~10x per day)



- John Schulman was a co-founder of OpenAI
- InstructGPT (first large-scale RLHF) used PPO to get a GPT model to obey human intent (crucial to get a language model to act as a chatbot)
- ChatGPT is a successor to InstructGPT

But...

Implementation Matters

Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO

Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry; ICLR 2020

Implementation Details

- Value function clipping
- Reward scaling
- Orthogonal initialization and layer scaling
- Adam learning rate annealing
- Reward clipping
- Observation normalization
- Observation clipping
- Hyperbolic tan activations
- Global gradient clipping

"PPO's marked improvement over TRPO (and even stochastic gradient descent) can be largely attributed to these optimizations."

Andrychowicz M, Raichuk A, Stańczyk P, et al. **What Matters In On-Policy Reinforcement Learning? A Large-Scale Empirical Study**. *arXiv*. Preprint posted online June 10, 2020:arXiv:2006.05990.
doi:[10.48550/arXiv.2006.05990](https://doi.org/10.48550/arXiv.2006.05990)

The 37 Implementation Details of Proximal Policy Optimization

Part III

Actor-Critic

Actor-Critic

Actor-Critic

Which should we learn? A , Q , or V ?

$$\nabla U(\theta) = E_{\tau} \left[\sum_{k=0}^d \nabla_{\theta} \log \pi_{\theta}(a_k | s_k) \underbrace{\gamma^k (r_k + \gamma V_{\phi}(s_{k+1}) - V_{\phi}(s_k))}_{\text{temporal difference residual}} \right]$$

$$l(\phi) = E \left[(V_{\phi}(s) - V^{\pi_{\theta}}(s))^2 \right]$$

↑ estimate with
reward to go
from sims

Generalized Advantage Estimation

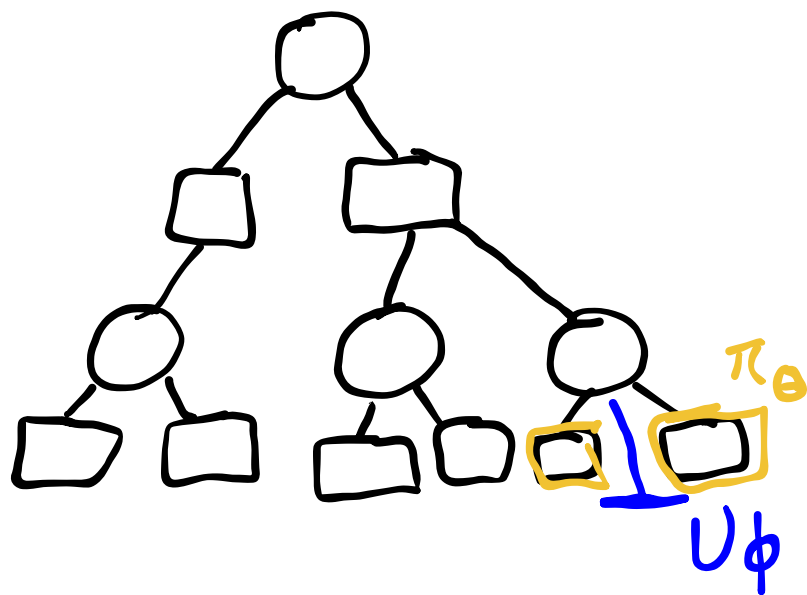
Recap

Alpha Zero: Actor Critic with MCTS

1. Use π_θ and U_ϕ in MCTS
2. Learn π_θ and U_ϕ from tree

$$\ell(\theta) = -\mathbb{E}_s \left[\sum_a \pi_{\text{MCTS}}(a | s) \log \pi_\theta(a | s) \right]$$

$$\pi_{\text{MCTS}}(a | s) \propto N(s, a)^\eta$$



$$\ell(\phi) = \frac{1}{2} \mathbb{E}_s \left[(U_\phi(s) - U_{\text{MCTS}}(s))^2 \right]$$

$$U_{\text{MCTS}}(s) = \max_a Q(s, a)$$

$$a = \arg \max_a Q(s, a) + c \pi_\theta(a | s) \frac{\sqrt{N(s)}}{1 + N(s, a)}$$